CS 422 Assignment 3: TCP Congestion Control (Option 1 — Kernel Module)

Joseph Getachew, Kushagr Khanna, Isaac Hallman

October 21, 2026

Repository: https://github.com/krash3125/cs422-assignment-2 (branch `assignment-3`). Numbers are from one run of `./run_a3.sh`, in `results_a3/20261024-151559/`.

1. How QAIMD updates the congestion window

`tcp_qaimd.c` implements the QAIMD design from Assignment 2 Q3(b): it anchors `cwnd` to the estimated BDP (`btl_bw × min_rtt`) and keeps the standing queue small. It registers as CC `"qaimd"` (`tcp_qaimd_ops`) and uses the `.cong_control` hook, so it owns every `cwnd` update; the Python client selects it with `setsockopt(TCP_CONGESTION, "qaimd")` and reads it back (`cc_used = qaimd` in all 94 successful tests).

Per ACK (`qaimd_main`) it tracks **RTT** (`min_rtt` over a 10 s window, and the per-round minimum), **throughput** (`btl_bw`, a windowed-max delivery rate), and a **loss** proxy (per-round retransmit/delivered counts). Queue `q = round_min_rtt − min_rtt`; a loss is *congestive* only if `q > q_thr` or the round loss ratio > 2%.

Per round (`qaimd_update_Round`):

- **Startup:** double `cwnd` until congestive loss, a sustained queue, or a bandwidth plateau; then drop to the BDP.
- **Avoid:** congestive loss → `cwnd ← 0.7·cwnd`; queue over threshold → proportional back-off; below BDP → catch up by `(bdp−cwnd)/4`; else `+1 MSS`.
- **RTO:** `cwnd = 4`, restart in startup capped at the last BDP.

2. Initial state, parameters, and changes from the pseudocode

Parameter	Value
initial state / cwnd	`STARTUP` / 10 MSS
`QAIMD_BETA`	0.7 (multiplicative decrease)
`QAIMD_LOSS_TOL`	2.0%
`QAIMD_MIN_CWND`	4 MSS
`q_thr`	clamp(0.15·min_rtt, 3–15 ms)
`QAIMD_PACE_GAIN`	1.1×

Changes from the pseudocode: (1) fixed-point integer math (kernel forbids FP); (2) loss ratio in segments (`total_retrans/delivered`), not bytes; (3) `btl_bw` is a time-windowed max, not a ring buffer (104-byte state limit); (4) `min_rtt` refreshed via the ProbeRTT drain, not an overwrite; (5) delay back-off capped at 30%; (6) uses the 2-argument `cong_control(sk, rs)` hook of the 6.8 kernel.

3. Kernel environment, build, and reproduction

- **Kernel:** `6.8.0-147-generic` (aarch64), Ubuntu 24.04.5, in a Multipass VM on macOS. Requires `CONFIG_TCP_CONG_ADVANCED=y` and `tcp_bbr`. Recorded in `metadata.json`.

- **Build & load:**

```
sudo apt-get install -y build-essential linux-headers-$(uname -r) python3-numpy python3-pandas python3-matpl
make            # builds tcp_qaimd.ko (KDIR=/lib/modules/$(uname -r)/build)
make load       # insmod; available CC becomes: reno cubic bbr qaimd
```

- **Run:** `sudo ./run_a3.sh` → `results_a3/<RUN_ID>/{samples.csv, tests.jsonl, tables, plots}`.

4. Method

8 of 10 public iperf3 servers were selected (`destinations.json`); each tested with **qaimd, cubic, reno, bbr** for **10 s**, sampled every **200 ms** (loss signal: retransmits, RTO events, `ca_state`). **3 repetitions**, algorithm order rotated each rep (rep1 qaimd→cubic→reno→bbr; rep2 cubic→reno→bbr→qaimd; rep3 reno→bbr→qaimd→cubic). **94 of 96 tests succeeded.** Representative destination: **`84.17.41.11`** (Seattle, min RTT ~48 ms).

5. Results

Aggregate over all destinations/reps (`tables/a3_algo_summary.md`):

CC	runs	goodput mean (Mbps)	goodput p95	srtt median (ms)	RTT inflation	retrans %
qaimd	23	103.5	186.8	67.8	1.09	0.24
cubic	24	139.6	254.6	114.3	1.61	0.31
reno	24	144.2	253.8	104.2	1.54	0.63
bbr	23	157.6	283.0	114.5	1.68	0.43

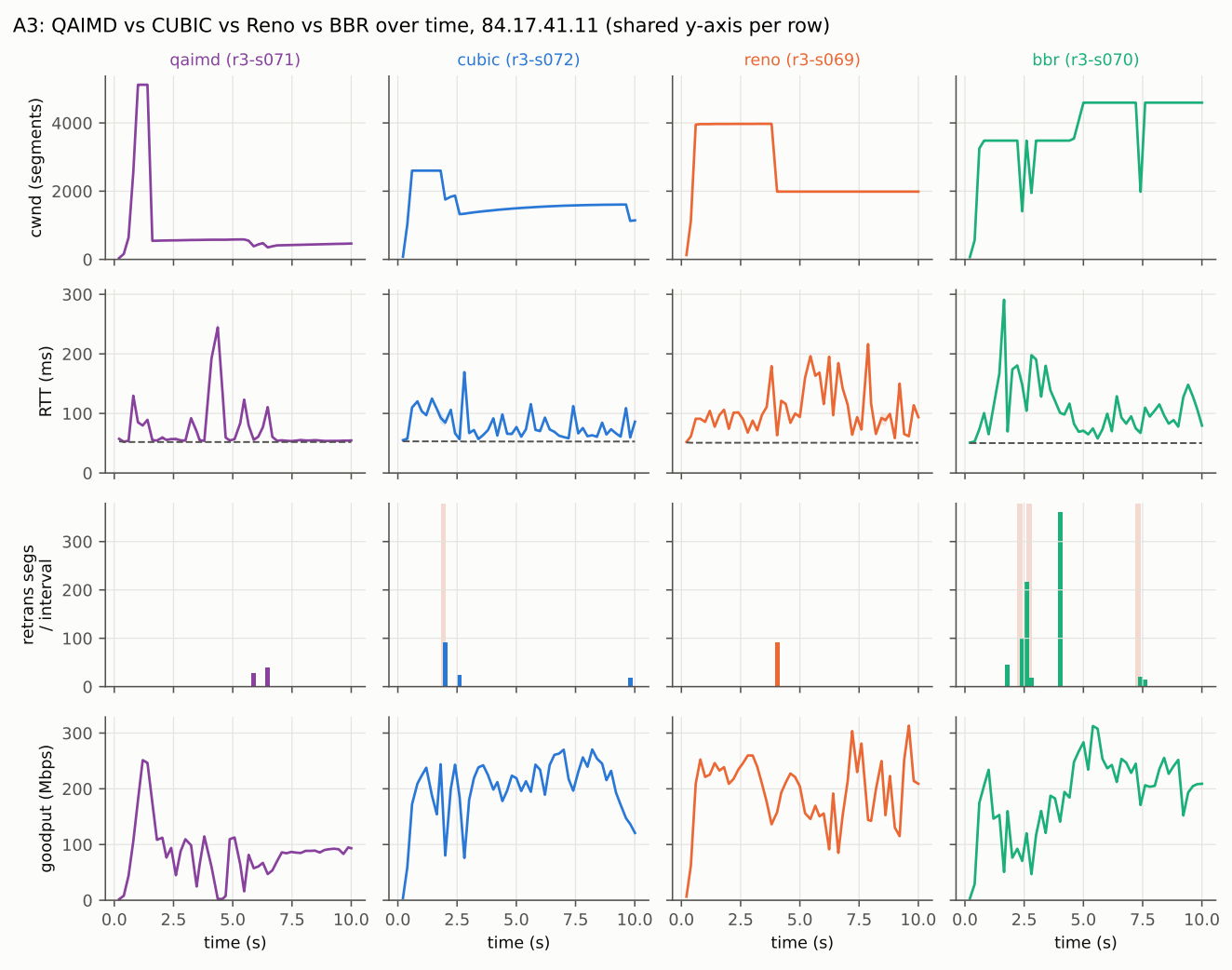


Figure 1: Time series: cwnd, RTT, loss, goodput — all four algorithms, Seattle.

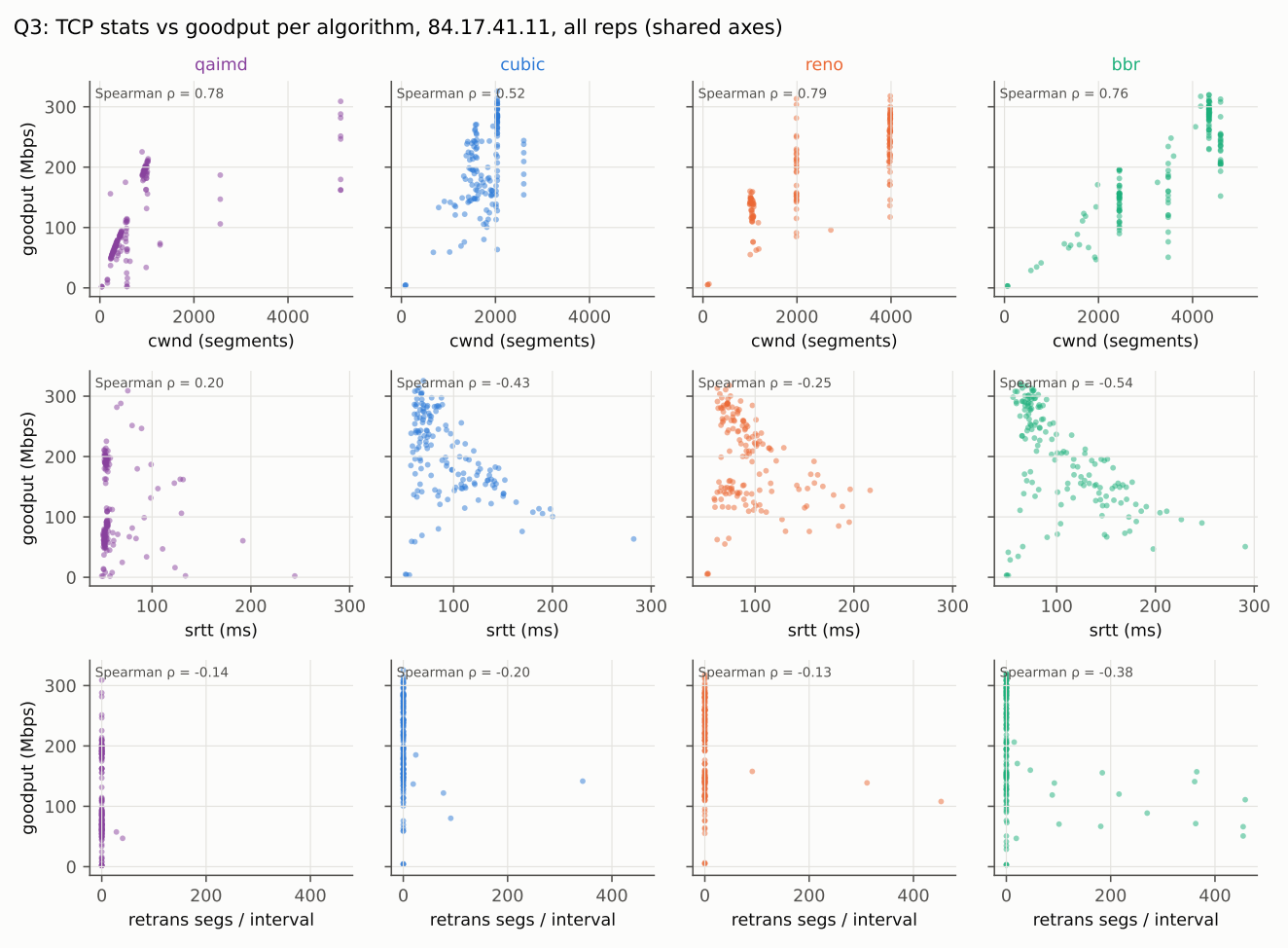


Figure 2: Scatter: cwnd, srtt, retransmits vs goodput — all four algorithms, Seattle.

Observed differences. CUBIC, Reno and BBR push `cwnd` past the BDP, so the excess sits in the bottleneck queue: higher goodput (140–158 Mbps) but RTT inflation 1.5–1.7×. QAIMD holds `cwnd` near the BDP, giving the lowest srtt (inflation 1.09×) and lowest retransmit rate (0.24%) at ~30% less throughput — it has the lowest srtt on 7 of 8 destinations (`tables/q3_winners.md`). On the long, thin paths (Sydney, Singapore) no algorithm builds a queue (inflation about 1.0), so QAIMD's latency advantage disappears and its conservatism costs throughput.

6. Use of AI tools

We used Claude Code to translate the QAIMD pseudocode into a kernel module, check it against the 6.8 `tcp_congestion_ops` interface, build and load it in the Multipass VM, and draft this report. We verified it by loading the module, confirming `qaimd` is selected on the socket, and checking server bytes match `tcpi_bytes_acked` to 0.04%.